

Python으로 Nav2 다뤄보기



Contents

1 Nav2 Python으로 접근

1.1 오늘의 목표

1.2 오늘을 위한 사전 준비

1.3 rviz에서 제자리를 잘 잡았는지 확인하고

1.4 여기가 x=0, y=1 지점

1.1 Navigation의 goal_pose로 로봇 구동

1.1.1 일단 연습 코드 위치

1.1.2 BasicNavigator의 action을 이용한 구동

1.1.3 코드 분석 1: Import와 초기화

1.1.4 쿼터니언 변환

1.1.5 goal 데이터 정의

1.1.6 action 서버중에 navigate_to_pose가 있다

1.1.7 goToPose

1.1.8 Nav2 내부 데이터 전달

1.1.9 진행 상황을 모니터링 하고 싶다면

1.1.10 최종결과를 알려주는 코드

1.1.11 main 함수를 호출하기

1.1.12 실행해보기

1.1.13 이렇게 움직이기 시작한다

1.2 정해진 시간에 도착하지 못하면 멈추게 해보자

1.2.1 코드 공유

1.2.2 타임아웃을 수동으로 만들어보자

1.2.3 먼저 입력으로 시간 정보도 수집

1.2.4 일정 시간이 지나면 cancelTask

1.2.5 실행해보면

1.2.6 가다가 멈춘걸 확인할 수 있다

1.3 goal_pose 토픽으로도 움직여보자

1.3.1 코드 공유

1.3.2 토픽으로 목표를 하달하자

1.3.3 goal_pose 토픽 발행 준비

1.3.4 메시지 타입 잘 만들고

1.3.5 메시지 발행

1.3.6 알아야할 함수 → `node.publisher.get_subscription_count()`

1.3.7 그래서 구독할 준비가 된건지 확인 (사용상 주의할 것) -이 코드는 안해도 되지만

1.3.8 그리고 한 10번 발행

1.3.9 명령

1.3.10 잘 간다

1.4 잠깐, 액션과 토픽

1.4.1 액션과 토픽 방식의 비교

1.4.2 실습 코드 및 명령 또 강조

Python으로 Nav2 다뤄보기

Exported from Confluence · <https://pinkwink.atlassian.net>

Nav2 Python으로 접근

오늘의 목표



액션 방식 제어

BasicNavigator API를 사용하여 로봇에게 정확한 목적지 이동 명령(goToPose)을 내리는 방법을 배웁니다.



타임아웃 처리

로봇이 지정된 시간 내에 도착하지 못할 경우, cancelTask()를 통해 안전하게 작업을 중단하는 로직을 구현합니다.



토픽 방식 제어

/goal_pose 토픽을 직접 발행하여 퍼블리셔(Publisher) 기반으로 로봇을 제어하는 간단한 방법을 알아봅니다.



실습 및 정리

실제 시뮬레이션 환경에서 코드를 실행해보고, 주요 함수 정리 및 GitHub 저장소 정보를 공유합니다.

오늘을 위한 사전 준비

Step 1 Gazebo 시뮬레이터 실행

```
$ ros2 launch pinky_gz_sim launch_sim.launch.xml
```

가상 로봇 환경(Gazebo)을 실행하여 시뮬레이션을 준비합니다.

Step 2 Navigation2 스택 실행

```
$ ros2 launch pinky_navigation bringup_launch.xml map:=<my_map.yaml> use_sim_time:=true
```

자신의 맵 파일 경로를 입력하고, 시뮬레이션 시간을 사용하도록 설정해야 합니다.

Step 3 RViz 시각화 도구 실행

```
$ ros2 launch pinky_navigation nav2_view.launch.xml
```

로봇의 현재 위치, 맵, 경로 계획 등을 시각적으로 확인하기 위해 RViz를 실행합니다.

rviz에서 제자리를 잘 잡았는지 확인하고


```
for_ROS2_study / R2R 심화편 Python으로 ROS2 navigation pkg 사용해보기 / nav2_send_goal_basic.py
PinkWink add nav2 python a35fefa · 1 minute ago History
Code Blame 86 Lines (68 loc) · 2.34 KB
1 #!/usr/bin/env python3
2 import math
3 import time
4 import sys
5
6 import rclpy
7 from geometry_msgs.msg import PoseStamped
8 from nav2_simple_commander.robot_navigator import BasicNavigator, TaskResult
9
10
11 def quaternion_from_yaw(yaw_rad: float):
12     """yaw(rad) -> (z, w) 쿼터니언 변환"""
13     z = math.sin(yaw_rad / 2.0)
14     w = math.cos(yaw_rad / 2.0)
15     return z, w
16
```

BasicNavigator의 action을 이용한 구동



BasicNavigator

Nav2가 제공하는 Python API 클래스로, 복잡한 액션 클라이언트 구현 없이 직관적인 함수 호출로 네비게이션 기능을 사용할 수 있습니다.



Action 클라이언트

goToPose() 함수 호출 시 내부적으로 'NavigateToPose' 액션 서버에 목표(Goal)를 전송하며 네비게이션을 시작합니다.



내부 동작 원리

요청을 받은 Nav2는 Behavior Tree(BT)에 따라 Planner(경로 생성), Controller(로컬 제어), Recovery(회복)를 순차적으로 수행합니다.



피드백 및 결과

이동 중에는 남은 거리 등의 피드백을 지속적으로 수신하며, 도착 성공 여부나 실패 원인(결과)을 확인할 수 있습니다.

코드 분석 1: Import와 초기화

Step 1 필수 모듈 Import

```
import rclpy
from geometry_msgs.msg import PoseStamped
from nav2_simple_commander.robot_navigator import BasicNavigator
```

BasicNavigator는 Nav2에서 제공하는 파이썬 API로, 복잡한 액션 서버 통신을 간단한 함수 호출로 추상화해줍니다.

Step 2 Navigator 초기화

```
rclpy.init()
# Nav2 API 인스턴스 생성
navigator = BasicNavigator()
```

ROS2 통신을 초기화한 후, navigator 객체를 생성합니다. 이 객체를 통해 로봇에게 이동 명령(goToPose)을 내리고 상태를 확인할 수 있습니다.

쿼터니언 변환

직접 구현하는 변환 함수

ROS2에서는 방향을 Quaternion(x, y, z, w)으로 표현합니다. 도(Degree) 단위를 라디안(Radian)으로 바꾼 뒤 변환합니다.

```
import math

def yaw_deg_to_quat(deg: float):
    # Degree를 Radian으로 변환
    yaw = math.radians(deg)

    # Z축 회전(Yaw)만 고려한 Quaternion 계산
    return (
        0.0,
        0.0,
        math.sin(yaw/2.0), # z
        math.cos(yaw/2.0) # w
    )
```

왜 Quaternion을 쓰나요?

집벌 락(Gimbal Lock) 현상을 방지하고 3차원 공간에서의 회전을 수학적으로 안정되게 표현하기 위해 사용합니다. 2D 평면 주행 로봇은 Z축 회전(Yaw)만 중요합니다.

라이브러리 사용 시 (tf_transformations)

```
from tf_transformations import quaternion_from_euler

# Roll, Pitch, Yaw 순서 (Radian)
qx, qy, qz, qw = quaternion_from_euler(
    0.0,
    0.0,
    math.radians(deg)
)
```

외부 라이브러리를 사용하면 3축 회전을 더 쉽게 처리할 수 있습니다.

goal 데이터 정의

nav2_send_goal_basic.py

Python 3.8+

```
# 목표 위치 및 자세 설정
x, y, yaw_deg = 0.0, 1.0, 90.0
q = yaw_deg_to_quat(yaw_deg)

goal = PoseStamped()
goal.header.frame_id = 'map'
goal.header.stamp = navigator.get_clock().now().to_msg()

goal.pose.position.x = x
goal.pose.position.y = y
goal.pose.orientation.z = q[2]
goal.pose.orientation.w = q[3]
```

좌표 설정

x, y 는 로봇이 도달할 2D 좌표(미터 단위)를 설정합니다. 시뮬레이션 환경의 map 프레임 기준으로 위치를 지정합니다.

방향(Orientation)

yaw 각도를 앞서 정의한 변환 함수를 통해 쿼터니언(z, w) 값으로 변환하여 자세를 설정합니다.

헤더(Header)

frame_id 는 'map'으로 지정하고, 현재 시간(stamp)을 포함하여 메시지의 기준과 시점을 명시합니다.

action 서버중에 navigate_to_pose가 있다

```
pw@pw:~/pinky_ws$ ros2 action list
/assisted_teleop
/backup
/compute_path_through_poses
/compute_path_to_pose
/drive_on_heading
/follow_gps_waypoints
/follow_path
/follow_waypoints
/navigate_through_poses
/navigate_to_pose
/spin
/wait
```

goToPose

```
# 액션 클라이언트 호출: 목표 지점으로 이동 시작
navigator.goToPose(goal_pose)

# 이 함수는 비동기(Non-blocking)로 동작하며,
# Nav2의 'NavigateToPose' 액션 서버에 목표를 전송합니다.
```

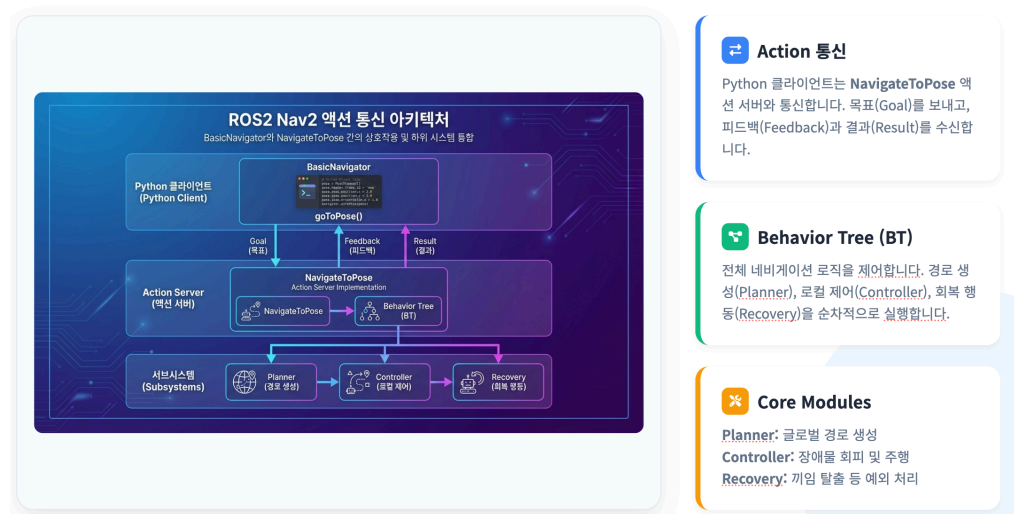
💡 핵심 포인트: goToPose()

단순한 함수 호출 같지만, 내부적으로는 **Action Client**가 생성되어 Nav2 스택 전체를 구동시키는 트리거 역할을 합니다.

🏗️ Nav2 내부 동작 흐름



Nav2 내부 데이터 전달



진행 상황을 모니터링 하고 싶다면

```
61 # 진행 모니터링
62 while not navigator.isTaskComplete():
63     feedback = navigator.getFeedback()
64     if feedback is not None:
65         print(
66             f"[Feedback] 남은 거리: {feedback.distance_remaining:.2f} m, "
67             f"경과 시간: {feedback.navigation_time.sec} s"
68         )
69     time.sleep(0.5)
```

최종결과를 알려주는 코드

```
71 # 최종 결과
72 result = navigator.getResult()
73 if result == TaskResult.SUCCEEDED:
74     print("[Result] 성공적으로 도착했습니다.")
75 elif result == TaskResult.CANCELED:
76     print("[Result] Goal이 취소되었습니다.")
77 elif result == TaskResult.FAILED:
78     print("[Result] Goal 수행에 실패했습니다.")
79 else:
80     print("[Result] 알 수 없는 상태:", result)
81
82 rclpy.shutdown()
```

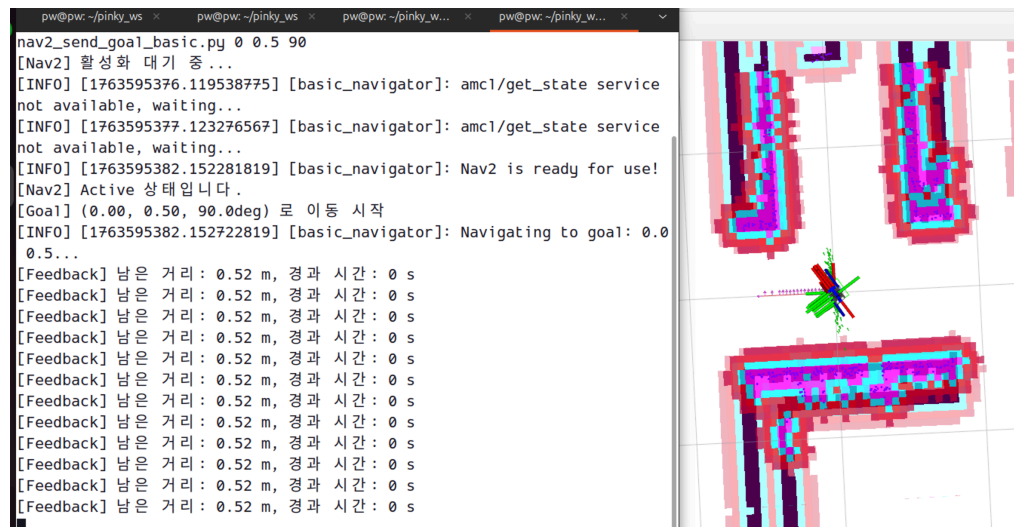
main 함수를 호출하기

```
84
85 if __name__ == "__main__":
86     main()
87
```

실행해보기

```
pw@pw:~/pinky_ws/src$
pw@pw:~/pinky_ws/src$ ls
nav2_send_goal_basic.py  pinky_pro
nav2_send_goal_cancel_code.py  topic_goal_pose_publisher.py
pw@pw:~/pinky_ws/src$
pw@pw:~/pinky_ws/src$ python3 nav2_send_goal_basic.py 0 0.5 90
```

이렇게 움직이기 시작한다

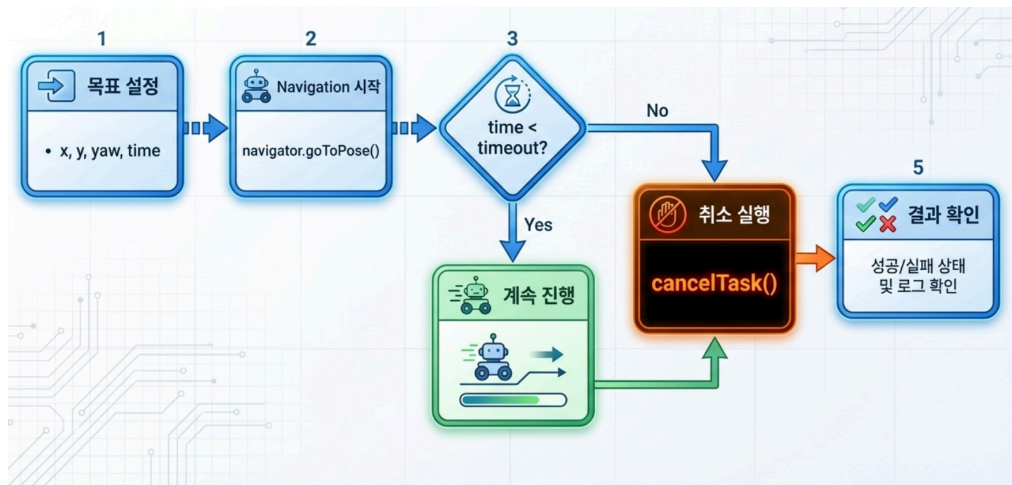


정해진 시간에 도착하지 못하면 멈추게 해보자

코드 공유

- https://github.com/PinkWink/for_ROS2_study/blob/main/R2R%20%EC%8B%AC%ED%99%94%ED%8E%B8%20Python%EC%9C%BC%EB%A1%9C%20ROS2%20navigation%20pkg%20

타임아웃을 수동으로 만들어보자



먼저 입력으로 시간 정보도 수집

```

28     try:
29         x = float(sys.argv[1])
30         y = float(sys.argv[2])
31         yaw_deg = float(sys.argv[3])
32         CANCEL_AFTER_SEC = float(sys.argv[4])
33     except ValueError:
34         print("Error: x, y, yaw_deg, time must be numbers")
35     return
  
```

일정 시간이 지나면 cancelTask

```

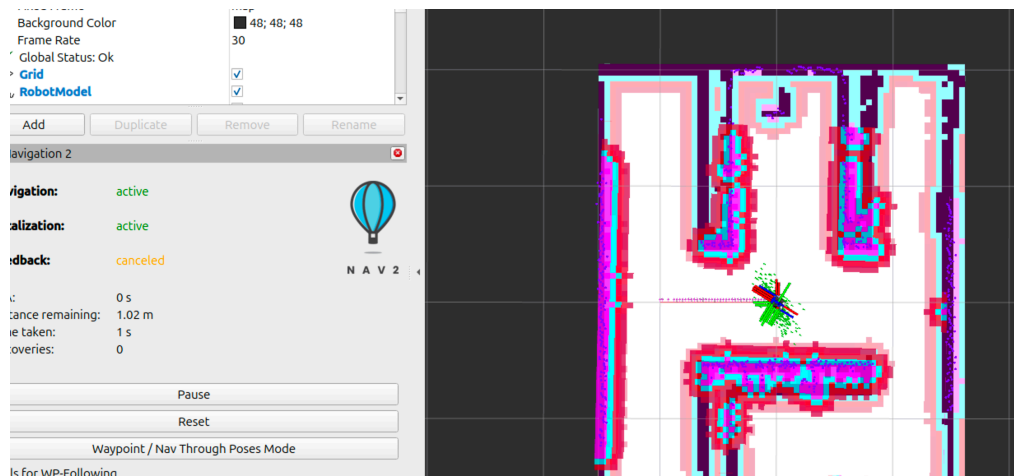
65     while not navigator.isTaskComplete():
66         if feedback is not None:
67             if elapsed > CANCEL_AFTER_SEC:
68                 print(f"[Action] {CANCEL_AFTER_SEC}초 초과 → Goal 취소 요청")
69                 navigator.cancelTask()
70                 break
  
```

실행해보면

```

pw@pw:~/pinky_ws/src$ python3 nav2_send_goal_cancel_code.py 0 1 0 5
[Nav2] 활성화 대기 중 ...
[INFO] [1763704563.044696350] [basic_navigator]: Nav2 is ready for use!
[Nav2] Active 상태입니다.
[Goal] (0.00, 1.00, 0.0deg) 로 이동 시작
[INFO] [1763704563.045446475] [basic_navigator]: Navigating to goal: 0.0 1.0...
[Feedback] 남은 거리: 1.02 m, 경과 시간: 0 s (실시간 0.1 s)
[Feedback] 남은 거리: 1.02 m, 경과 시간: 0 s (실시간 0.7 s)
[Feedback] 남은 거리: 1.02 m, 경과 시간: 0 s (실시간 1.3 s)
[Feedback] 남은 거리: 1.02 m, 경과 시간: 0 s (실시간 1.9 s)
[Feedback] 남은 거리: 1.02 m, 경과 시간: 0 s (실시간 2.5 s)
[Feedback] 남은 거리: 1.02 m, 경과 시간: 0 s (실시간 3.1 s)
[Feedback] 남은 거리: 1.02 m, 경과 시간: 0 s (실시간 3.7 s)
[Feedback] 남은 거리: 1.02 m, 경과 시간: 0 s (실시간 4.3 s)
[Feedback] 남은 거리: 1.02 m, 경과 시간: 0 s (실시간 4.9 s)
[Feedback] 남은 거리: 1.02 m, 경과 시간: 1 s (실시간 5.5 s)
[Action] 5.0초 초과 → Goal 취소 요청
[INFO] [1763704568.568729227] [basic_navigator]: Canceling current task.
[Result] 알 수 없는 상태: TaskResult.UNKNOWN
[Result] TaskResult = TaskResult.UNKNOWN
  
```

가다가 멈춘걸 확인할 수 있다



goal_pose 토픽으로도 움직여보자

코드 공유

- https://github.com/PinkWink/for_ROS2_study/blob/main/R2R%20EC%8B%AC%ED%99%94%ED%8E%B8%20Python%EC%9C%BC%EB%A1%9C%20ROS2%20navigation%20pkg%20for%20WP-Following

토픽으로 목표를 하달하자

Goal Pose 발행

/goal_pose 토픽에 PoseStamped 메시지를 직접 Publish하여 네비게이션을 시작합니다.

단방향 통신

Action과 달리 피드백이나 결과를 기다리지 않는 Fire-and-Forget 방식입니다.

구독자 확인 필수

Nav2가 토픽을 듣고 있는지 get_subscription_count()로 확인 후 발행해야 합니다.

신속한 명령

복잡한 클라이언트 설정 없이 간단한 코드로 즉각적인 이동 명령을 전송할 수 있습니다.

goal_pose 토픽 발행 준비

```

17 class GoalPosePublisher(Node):
18     """
19     /goal_pose 토픽으로 PoseStamped를 Publish하는 예제
20     """
21
22     def __init__(self):
23         super().__init__("pinky_goal_pose_publisher")
24         self.publisher = self.create_publisher(PoseStamped, "/goal_pose", 10)
25         self.get_logger().info("goal_pose 퍼블리셔 준비 완료")

```

메시지 타입 잘 만들고

```

17 class GoalPosePublisher(Node):
27     def make_msg(self, x: float, y: float, yaw_deg: float) -> PoseStamped:
28         msg = PoseStamped()
29         msg.header.frame_id = "map"
30         msg.header.stamp = self.get_clock().now().to_msg()
31
32         msg.pose.position.x = float(x)
33         msg.pose.position.y = float(y)
34
35         yaw_rad = math.radians(yaw_deg)
36         msg.pose.orientation.z, msg.pose.orientation.w = quaternion_from_yaw(yaw_rad)
37
38         return msg

```

메시지 발행

```

17 class GoalPosePublisher(Node):
40     def send_goal_pose(self, x: float, y: float, yaw_deg: float):
41         msg = self.make_msg(x, y, yaw_deg)
42         self.publisher.publish(msg)
43         self.get_logger().info(
44             f"goal_pose 전송: x={x:.2f}, y={y:.2f}, yaw={yaw_deg:.1f} deg"
45         )

```

알아야 할 함수 → node.publisher.get_subscription_count()

- 이 퍼블리셔가 publish하는 토픽을 현재 DDS 네트워크에서 구독하는 subscriber 노드 수를 반환
- 반환 값은 정수(int)
- Nav2가 /goal_pose 를 구독하는 구조라면 → 1
- 구독자가 아무도 없으면 → 0

그래서 구독할 준비가 된건지 확인 (사용상 주의할 것) - 이 코드는 안해도 되지만

```

68 # ---- 1) subscriber 불을 때까지 기다리기 ----
69 wait_sec = 0.0
70 timeout = 2.0 # 최대 2초 기다림
71
72 while node.publisher.get_subscription_count() == 0 and wait_sec < timeout:
73     node.get_logger().info(
74         f"subscriber 대기 중... (현재 {node.publisher.get_subscription_count()}명, {wait_sec}초)"
75     )
76     rclpy.spin_once(node, timeout_sec=0.1)
77     wait_sec += 0.1
78
79 node.get_logger().info(
80     f"최종 subscriber 수: {node.publisher.get_subscription_count()}"
81 )

```

그리고 한 10번 발행

```

48 def main():
83     # ---- 2) goal_pose 여러 번 발행 ----
84     publish_count = 10
85     for i in range(publish_count):
86         node.send_goal_pose(x, y, yaw_deg)
87         rclpy.spin_once(node, timeout_sec=0.1)
88         time.sleep(0.1)

```

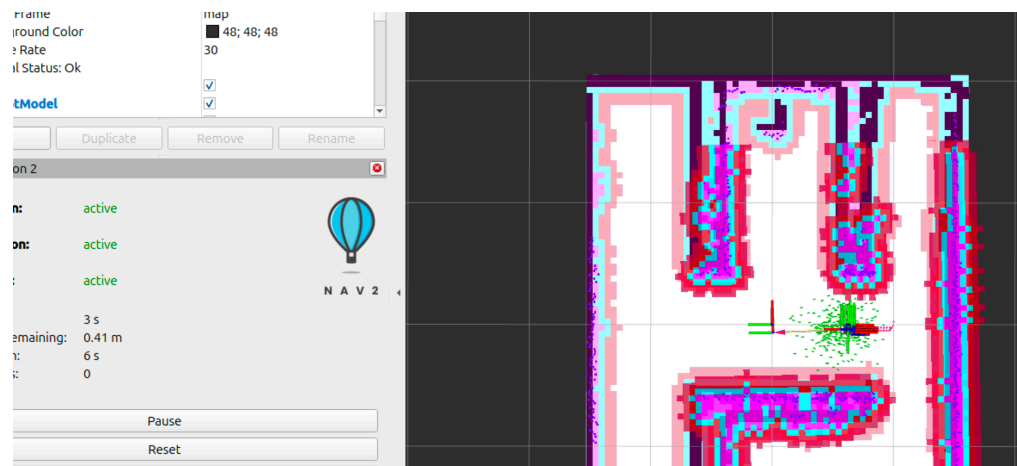
명령


```

jw@pw:~/pinky_ws/src$ python3 topic_goal_pose_publisher.py 0 -1 0
[INFO] [1763705124.630525701] [pinky_goal_pose_publisher]: goal_pose 퍼블리셔 준비 완료
[INFO] [1763705124.630772451] [pinky_goal_pose_publisher]: 최종 subscriber 수 : 0
[INFO] [1763705124.631014743] [pinky_goal_pose_publisher]: goal_pose 전송 : x=0.00, y=-1.00
, yaw=0.0 deg
[INFO] [1763705124.832006618] [pinky_goal_pose_publisher]: goal_pose 전송 : x=0.00, y=-1.00
, yaw=0.0 deg
[INFO] [1763705125.035665534] [pinky_goal_pose_publisher]: goal_pose 전송 : x=0.00, y=-1.00

```

잘 간다



잠깐, 액션과 토픽

액션과 토픽 방식의 비교

Nav2 제어 방식 비교

액션 방식 (Action)

- BasicNavigator 사용
- goToPose() 호출
- 피드백 수신 가능
- 결과 확인 가능
- 양방향 통신

토픽 방식 (Topic)

- Publisher 생성
- goal_pose 토픽 발행
- 단방향 전송
- 간단한 구현
- 빠른 명령

1. 액션 방식 (Action)

- ✓ 양방향 통신: 목표 전송, 피드백, 결과 수신 가능
- ✓ 신뢰성 높음: 작업 상태(성공/실패/취소) 확인 가능
- ✓ 제어 용이: 실행 중 취소(Cancel) 등의 제어가 가능

2. 토픽 방식 (Topic)

- ✓ 단방향 통신: 목표 위치만 던져주고 끝 (Fire & Forget)
- ✓ 구현 간단: 복잡한 클라이언트 설정 없이 퍼블리셔만 필요
- ✓ 빠른 명령: 오버헤드가 적어 단순 이동 명령에 유리

실습 코드 및 명령도 강조



PinkWink / for_ROS2_study

본 강의에서 사용된 모든 예제 코드와 설정 파일은 아래 GitHub 저장소에서 확인하실 수 있습니다.

github.com/PinkWink/for_ROS2_study



nav2_send_goal_basic.py

기본 Navigation 제어 예제



nav2_send_goal_cancel_code.py

타임아웃 및 취소 기능 구현



topic_goal_pose_publisher.py

토픽 방식 제어 예제



PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

https://www.youtube.com/@pinklab_studio